

Contrôle terminal 2022-23 - Session 1

Le barème est donné uniquement à titre indicatif.
Temps : 2h.

1 Entrées - Sorties (10 pts)

Documents autorisés : documents de cours.

1.1 Cours (2 pts)

1. Expliquez le cheminement détaillé du signal lors de la commande ci-dessous en considérant que la broche 10 du GPIO A est connectée sur une LED. (1 pts)

`GPIOA_BSRR |= 1 << 10;`

- a) Le processeur écrit sur le bus $1 << 10$ à l'adresse `GPIOA_BSRR`.
 - b) Le circuit BSRR transforme ce signal en mettant à 1 le bit 10 de `GPIOA_ODR`.
 - c) Ce bit est propagé dans le GPIO sous forme d'un signal de 3.3V qui se retrouve sur la broche de sortie PA10.
 - d) Les 3.3V viennent alimenter la LED qui s'allume.
2. La broche 4 du GPIO D est connecté sur un bouton poussoir et cette même broche est configurée avec EXTI (ligne 4) pour délivrer une interruption sur front montant. Expliquez ce qui se passe dans le micro-contrôleur quand le bouton est appuyé. (1 pts)
 - a) Le contact est fait dans le bouton poussoir et une tension apparaît sur la broche PD4.
 - b) Le front montant apparaissant déclenche l'activation de l'interruption EXTI4 dans EXTI et le signal propagé vers NVIC.
 - c) Dans NVIC, s'il n'y a déjà une interruption en cours priorité plus grande, on attend la fin de cette interruption.
 - d) Le signal est propagé vers le coeur d'exécution qui déclenche le démarrage de la fonction de gestion de l'interruption.

1.2 Exercice (8 pts)

Dans cet exercice, nous allons gérer la vitesse d'un petit véhicule motorisé. Le moteur est à courant continu et sera piloté par un signal PWM (période 20ms, broche PA6, TIM3 CH2, AF2). Pour contrôler la vitesse, le conducteur utilise une pédale d'accélération connectée à un capteur de force. Ce dernier produit une tension proportionnelle à la force exercée. Cette tension est mesurée par un ADC (ADC1, IN3, PA3).

1. Faire un schéma illustrant le circuit en faisant apparaître GPIO, timers, moteurs et pédales. (1 pts)
2. Calculer le meilleur diviseur et la période pour le timer du moteur (TIM3 connecté sur APB1 à 42Mhz). (1 pts)

$$\frac{2^{16} \times D}{MCK} = \frac{20}{1000} \iff D = \frac{20 \times MCK}{1000 \times 2^{16}} = \frac{20 \times 42\,000\,000}{1000 \times 65536} = 12,81$$

3. Écrire la fonction `init_motor()` qui permet d'initialiser le moteur. (1,5 pts)

```
#define PSC 14
#define PERIOD ( APB1_MCK * 20 / 1000 / PSC)

void init_motor() {
    GPIOA_MODER = SET_BITS(GPIOA_MODER, 6*2, 2, GPIOA_MODER_ALT);
    GPIOA_AFRL = SET_BITS(GPIOA_AFRL, 6*4, 4, 2);

    TIM3_CR1 = 0;
    TIM3_PSC = PSC - 1;
    TIM3_ARR = PERIOD;

    TIM3_CCMR1 = TIM_OC2_PWM1;
    TIM3_CCER = TIM_CC2E;
    TIM3_CCR2 = 0;

    TIM3_EGR = TIM_UG;
    TIM3_SR = 0;
    TIM3_CR1 = TIM_CEN | TIM_ARPE;
}
```

4. Écrire la fonction `set_motor(x)` qui permet de choisir la vitesse du moteur avec x un pourcentage de la période. (1 pts)

```
void set_motor(int x) {
    TIM3_CCR2 = PERIOD * x / 100;
}
```

5. Écrire la fonction `init_accel()` qui permet d'initialiser l'ADC de la pédale d'accélération. (1 pts)

```

void init_accel() {
    GPIOA_MODER = SET_BITS(GPIOA_MODER, 3*2, 2, GPIOA_MODER_ANA);
    ADC1_CR1 = ADC_12b;
    ADC1_SQR3 = 3;
}

```

6. Écrire la fonction `get_accel()` qui lit et renvoie la valeur de la pédale d'accélération. (1 pts)

```

int get_accel() {
    ADC1_SR = 0;
    ADC1_CR2 = ADC_SWSTART;
    while ((ADC1_SR & ADC_EOC) == 0);
    return ADC1_DR;
}

```

7. Écrire la fonction `main()` qui pilote la vitesse du véhicule à une fréquence de 20Hz. Le comportement est le suivant (x est la valeur sur la pédale d'accélération) : (1,5 pts)
- Si $x < LO_TH$ alors la vitesse diminue de 5%.
 - Si $LO_TH \leq x < HI_TH$ alors la vitesse ne change pas.
 - Si $x \geq HI_TH$ alors la vitesse augmente de $(x - HI_TH) / (1024 - HI_TH) \times 100\%$.

```

int main() {
    init_motor();
    init_accel();

    // period timer
    TIM4_CR1 = 0;
    TIM4_PSC = 42 - 1;
    TIM4_ARR = APB1_MCK * 50 / 1000 / 32;
    TIM4_EGR = TIM_UG;
    TIM4_SR = 0;
    TIM4_CR1 = TIM_CEN;

    int speed = 0;
    while(1) {
        while ((TIM4_SR & TIM_UIF) == 0);
        TIM4_SR = 0;
        int x = get_accel();
        if (x < LO_TH)
            speed = speed * 0.95;
        else if (x >= HI_TH) {
            speed += (x - HI_TH) * PERIOD / (1024 - HI_TH) * 100;
            if (speed > 100)
                speed = 100;
        }
    }
}

```

```
    }  
    set_motor(speed);  
}  
}
```

2 FPGA (4 pts)

Documents autorisés : aucun.

2.1 Cours

1. Le bus AXI est dit de type *"Memory Mapped"* : que cela signifie-t-il ?
2. Le traitement d'un signal requiert un grand nombre d'opérations de multiplications et additions : en quoi un FPGA sera-t-il, ou non, plus efficace qu'un processeur ? qu'un GPU ??

2.2 Exercice

Une caméra est un dispositif de capture qui généralement dispose de deux interfaces :

1. interface destinée au contrôle
2. interface pour l'acquisition des données

Vous disposez ainsi d'un composant VHDL **'camera'** (fig. 1) que vous devez interfacer via une ou plusieurs interfaces AXI.

- [1] L'interface de contrôle du composant **'camera'** dispose des signaux suivants :
- Enable on/off, IR_mode on/off, Autofocus on/off,
 - White_balance $0 \rightarrow 9$, Exposure_mode $0 \rightarrow 15$,
 - Brightness $0 \rightarrow BRIGHT_MAX$ et Contrast $0 \rightarrow CONTRAST_MAX$
 - Resolution_mode [640x480, 800x600, 1024x768, 1280x1024]
 - les paramètres génériques *BRIGHT_MAX* et *CONTRAST_MAX* peuvent aller jusqu'à 100

[2] L'interface de données quant à elle permet à l'hôte de récupérer un flux de données, à savoir les images. Cette dernière n'ayant pas été implémentée, à vous de proposer une solution cohérente eu égard au bus AXI.

Décrivez le bloc IP qui satisfait aux contraintes ci-dessus énoncées en faisant apparaître la/les interfaces AXI présentes dans votre composant. Vous détaillerez les différents registres utilisés et leur *mapping* dans l'espace mémoire du système hôte.

3 Énergie (3 pts)

Documents autorisés : aucun.

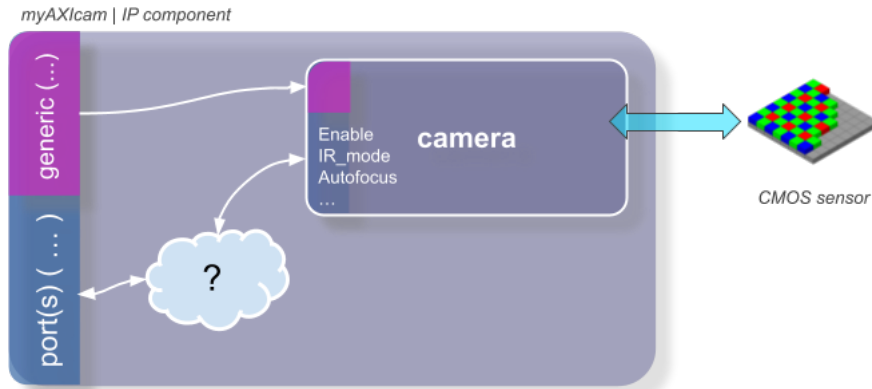


FIGURE 1 – VHDL 'camera' module to get AXI-enabled.

3.1 Cours

Explicitez le mécanisme du DVFS, son impact sur le temps, l'énergie consommée par une application.

3.2 Cours

Explicitez (avec des exemples) les liens et les différences entre charge du processeur et consommation énergétique d'un ordinateur.

4 Langages synchrones (3 pts)

Documents autorisés : aucun.

4.1 Exercice 1

On considère le noeud Lustre suivant :

```
node System(e_0, e_1 : int) returns (s_0 : int);
var cnt, v_0, v_1, v_2, v_3 : int;
c_0, c_1, c_2, in : bool;
let
  cnt = 0 -> if pre cnt = 7 then 0 else pre cnt + 1;
  c_0 = (cnt = 0) or (cnt = 1) or (cnt = 5);
  c_1 = (cnt = 2) or (cnt = 3);
  c_2 = (cnt = 1) or (cnt = 2) or (cnt = 4) or (cnt = 7);
  v_0 = current(node_f(e_0 when c_0));
  v_1,v_2 = if c_1 then current(node_g(e_1 when c_1, v_0 when c_1))
             else ((0,0) -> pre (v_1,v_2));
  v_3 = if c_2 then current(node_h( v_0 when c_2,
                                   v_1 when c_2,
                                   v_2 when c_2))
        else (0 -> pre v_3);
  s_0 = v_3;
tel
```

Détaillez le schéma de communications entre les différents sous-noeuds du noeud System décrit ci-dessus, pour ses 8 premiers instants d'activation $I_0, \dots, I_7$. Pour ce faire, dessinez une frise temporelle graduée par les instants $I_0, \dots, I_7$, dans laquelle chaque ligne correspond à un sous-noeud du système. Dans chaque ligne, marquez d'une barre verticale les instants auxquels le noeud correspondant est actif. Tracez ensuite des flèches entre les barres verticales pour expliciter les communications. On ne représentera pas les entrées et sorties du noeud System.

4.2 Exercice 2

Écrivez un noeud Lustre fEdge qui prend en entrée un signal booléen S et détecte les fronts descendants de S : la sortie de fEdge vaut 1 lorsqu'un front descendant est détecté, et 0 sinon. Le signal S est considéré égal à 0 au démarrage.